

Built-in Classes ko Extend karna

`Array`, `Map`, `Set` jaisi built-in classes ko bhi `extends` kar sakte hain.

javascript

```
class PowerArray extends Array {
  isEmpty() {
    return this.length === 0;
  }
}

let arr = new PowerArray(1, 2, 5, 10, 50);
arr.isEmpty(); // false

let filtered = arr.filter(item => item >= 10);
filtered.isEmpty(); // false
```

Ek mazedaar baat

`filter`, `map` jaise built-in methods naya array bhi `PowerArray` ka hi banate hain (normal `Array` ka nahi). Wo andar se `arr.constructor` use karte hain. Isliye result par bhi `isEmpty()` chalta hai.

`Symbol.species`

Agar chahte ho ki `map`/`filter` normal `Array` hi wapas de, to class me static getter `Symbol.species` likho jo wo constructor return kare:

javascript

```
class PowerArray extends Array {
  isEmpty() { return this.length === 0; }

  static get [Symbol.species]() {
    return Array;
  }
}
```

Ab `arr.filter(...)` normal `Array` dega, aur us par `isEmpty()` nahi milega. `Map` aur `Set` bhi aise hi chalte hain.

Built-ins me static inheritance nahi hoti

Normal classes me `extends` karne par static methods bhi inherit hote hain. Lekin built-in classes me aisa **nahi** hota.

Example: `Array` aur `Date` dono `Object` se inherit karte hain, to unke **objects** ko `Object.prototype` ke methods milte hain. Par `Array.[[Prototype]]` `Object` ko point nahi karta, isliye `Array.keys()` ya `Date.keys()` jaisa static method nahi milta.

Yaani sirf `Date.prototype` -> `Object.prototype` link hai. `Date` aur `Object` khud ek doosre se juda nahi.

Yaad rakho

- Built-ins ko extend kar sakte ho aur naye methods jod sakte ho.
- `map`/`filter` ka result same extended type ka hota hai.
- Type badalna ho to `Symbol.species`.
- Built-in classes ke static methods aapas me inherit nahi hote.